

Computer Vision: Assignment 2

Deep Learning for Computer Vision

Pablo Mesejo and Siham Tabik

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



Index

- Submission rules
- Assignment description
- Brief introduction to fastai

Index

- **Submission rules**
- Assignment description
- Brief introduction to fastai

Submission rules

- Only *.ipynb* files should be submitted (including code, results, analysis and discussion).
 - Don't upload the images!
- Don't write anything to disc/Drive.
- The templates structure must be respected.

Submission

- Deadline: 7 December
 - Please, plan well in advance, and make responsible/rational use of Google Colab.
 - Having Google Colab Pro does not imply infinite resources!
- Maximum score: 15 points
- Submission Site: <https://pradogrado2526.ugr.es/>
- **Explanation/discussion accompanying code and results is essential.**

Goals

- Learn how to implement **convolutional neural networks using fastai/PyTorch**.
- Understand the concepts of **feature extraction** and **fine-tuning**.
- Begin to familiarize yourself with **explainable AI**.
- This is an assignment oriented towards **image classification & regression using Deep Learning**.

Materials at your disposal

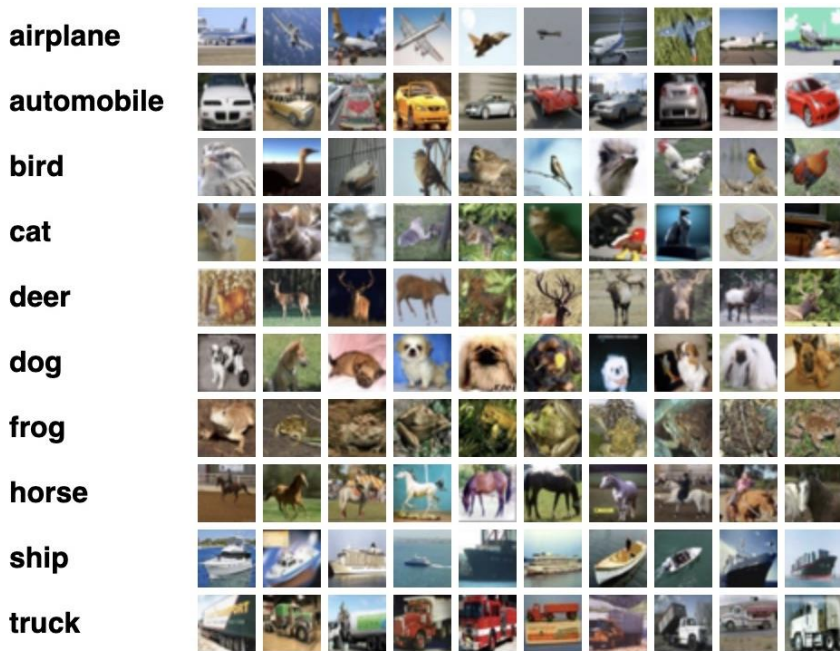
- This **introduction to assignment 2 and fastai**
- Templates:
P2_Ejs1y2_TEMPLATE.ipynb
P2_Ej3_TEMPLATE.ipynb
P2_Ej4_TEMPLATE.ipynb
- A help guide with codes and examples:
P2_HG.ipynb

Index

- Submission rules
- **Assignment description**
- Brief introduction to fastai

Exercise 1: BaseNet in CIFAR100 (5 points)

1. Create a simple model (called BaseNet)
2. Train and validate it with the (reduced) CIFAR100 dataset



Exercise 1: BaseNet in CIFAR100 (5 points)

Layer Type	Kernel Size (for convolutional layers)	Input Output dimension	Input Output channels (for convolutional layers)
Conv	7x7	32x32 32x32	3 10
Sigmoid	-	32x32 32x32	-
MaxPooling	2x2	32x32 16x16	-
Conv	7x7	16x16 10x10	10 15
Tanh	-	10x10 10x10	-
FC	-	???? 750	-
ReLU	-	750 750	-
FC	-	750 150	-
ReLU	-	150 150	-
FC	-	150 50	-

**Architecture
you have to
implement in
fastai**

Taking into account that this is a multiclass
classification problem, what is the most
natural/common choice for the activation
function and loss function?



Exercise 1: BaseNet in CIFAR100 (5 points)

- **Compare training in CPU vs GPU (0.5 pts)**

- ☐ CPU ☐ GPU A100 ☐ GPU L4 ☐ GPU T4
- ☐ TPU v6e-1 ☐ TPU v5e-1

- In case you want to learn more about GPUs:

- <https://jax-ml.github.io/scaling-book/gpus/>
- <https://sebastianraschka.com/teaching/pytorch-1h/>
- <https://www.jeremyjordan.me/distributed-training/>

Exercise 1: BaseNet in CIFAR100 (5 points)

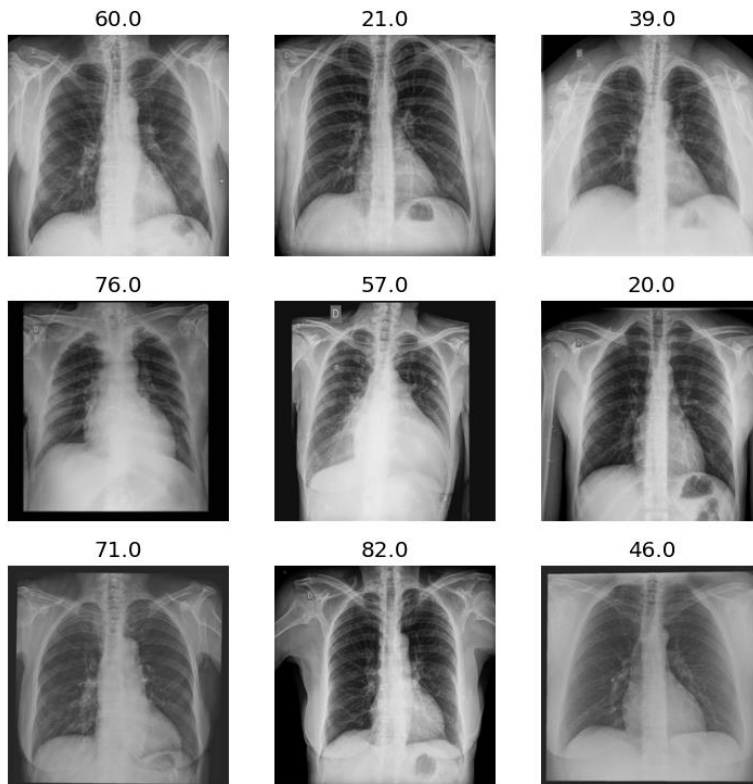
- FC layers force the input to have always the same size & add a lot of weights.
 - **How can we remove the FC layers from our model? Implement a new BaseNet model without FC layers (0.5 pts)**

Exercise 2: Improvement of the BaseNet model

(4 points)

- Once you have implemented and validated BaseNet, you should improve the network by means of those alternatives that you judge to be appropriate:
 - Data augmentation
 - Network depth augmentation
 - Batch normalization
 - Regularization
 - Dropout
 - Early-Stopping
 - Others?
 - You can try things you have seen in theory or from other sources.
 - Unleash your creativity and intuition.
 - Innovation, complexity and good use of PyTorch/fastai will be highly valued.
- Always remember to **justify your decisions** (it is not about testing for the sake of testing) and to **clearly show the final architecture**.

Exercise 3: Model transfer and fine-tuning with ResNet34 for the SPR X-Ray Age Prediction Challenge (4 points).



We start from a **model trained in a classification problem**, and we try to **apply it in a regression problem**.

Exercise 3: Model transfer and fine-tuning with ResNet34 for the SPR X-Ray Age Prediction Challenge (4 points).

1. Train from scratch the entire ResNet34 (0.75 pts)
2. Use ResNet34 as a feature extractor (1.25 pts)
 - Remove the final fully-connected (FC) layer of ResNet34, replace it by a FC layer of the dimensionality of the new problem, and **train the new weights of this FC layer** (while keeping frozen the remaining weights in the network).
3. Fine-tune the entire ResNet34 (1 pt)
4. Fine-tune the entire ResNet34, but posing the task as a classification problem (age classes $\in \{18, \dots, 89\}$) (1 pt)

Exercise 4: EfficientNet-B0 fine-tuning and explainable AI in Caltech-UCSD Birds-200-2011 (2 points).



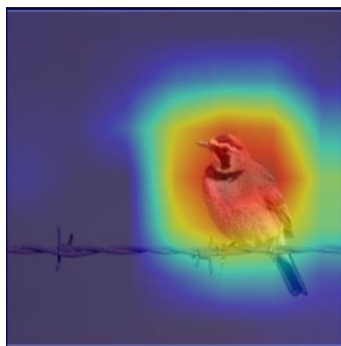
Exercise 4: EfficientNet-B0 fine-tuning and explainable AI in Caltech-UCSD Birds-200-2011 (2 points).

We have to fine-tune EfficientNet-B0 on this classification problem (0.5 pts), and then we **use Grad-CAM to visualize input image regions that are relevant for the prediction (1 pt).**

Very important references:

<https://jacobgil.github.io/pytorch-gradcam-book/introduction.html>

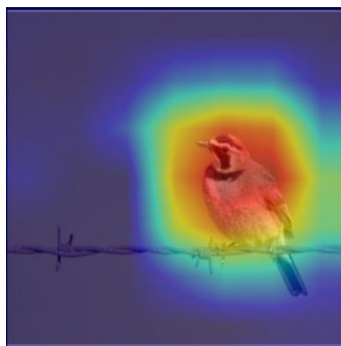
<https://github.com/jacobgil/pytorch-grad-cam>



Exercise 4: EfficientNet-B0 fine-tuning and explainable AI in Caltech-UCSD Birds-200-2011 (2 points).

We also have to answer two conceptual questions:

- **How EfficientNet-B0 was created? (0.25 pts)**
- **Grad-CAM limits (0.25 pts):** *Ghassemi et al. "The false hope of current approaches to explainable artificial intelligence in health care." The lancet digital health 3.11 (2021): e745-e750*



Index

- Submission rules
- Assignment description
- **Brief introduction to fastai**

Highly Recommended References

- **Book with Colab Notebooks:**
<https://github.com/fastai/fastbook>
- Course "Practical Deep Learning for Coders 2022"
(https://www.youtube.com/playlist?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU; <https://course.fast.ai/>;
academic year 2019: <https://course19.fast.ai/videos/>).
- Jupyter Notebooks of Jeremy Howard:  <https://www.kaggle.com/jhoward/code>

Founding researcher (fast.ai),
Distinguished Research
Scientist (University of San
Francisco), former President
and Chief Scientist (Kaggle)

Highly Recommended References

- **Book with Colab Notebooks:**
<https://github.com/fastai/fastbook>
- Course "Practical Deep Learning for Coders 2022"
(https://www.youtube.com/playlist?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU; <https://course.fast.ai/>;
academic year 2019: <https://course19.fast.ai/videos/>).

Note on these materials: The strategy of the book (notebooks and videos) is *top-down*: you start from the code, experiment with it, extract intuitions, and then analyze how it works and go deeper into the fundamentals.

Our framework: fastai

We use **fastai** (based on **PyTorch**).
Main reference: <https://docs.fast.ai/>

PyTorch Cheat Sheet: <https://www.mad.tf.fau.de/files/2019/07/pytorch-cheatsheet-en.pdf>

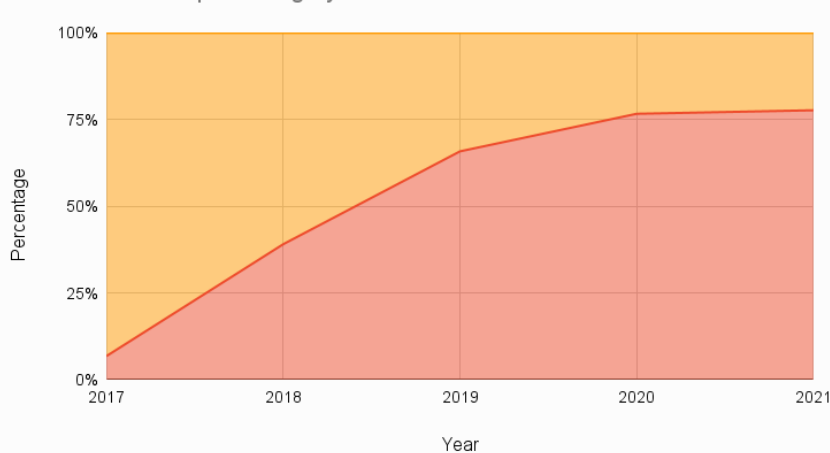
PyTorch documentation: <https://docs.pytorch.org/tutorials/index.html>

	Keras (TensorFlow API)	fastai (PyTorch API)
Strengths	• Very simple	• Uses PyTorch (probably the most popular DL tool today)
Weaknesses	• Possibly, less complete and flexible than fastai/PyTorch	• Possibly, longer learning curve • It has so many high-level functionalities, that you may not fully understand what is being done at lower level (e.g. test data normalization)

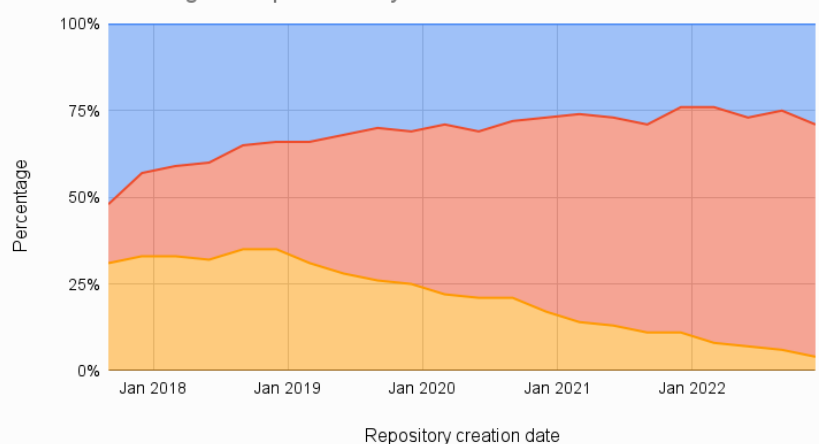
Our framework: fastai

We use **fastai** (based on **PyTorch**).
Main reference: <https://docs.fast.ai/>

Fraction of Papers Using PyTorch vs. TensorFlow



Percentage of Repositories by Framework



Our framework: fastai

We use **fastai** (based on **PyTorch**).
Main reference: <https://docs.fast.ai/>

When using fastai you have access to the full potential of PyTorch and new features. As a result, you do not have to write so much code.

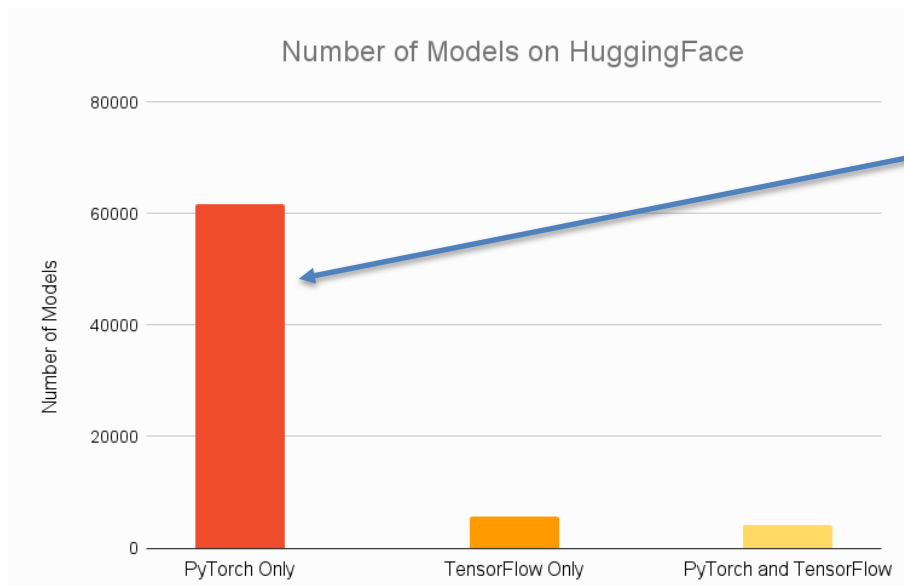
But, if you prefer to code using plain PyTorch, feel free to do it!!!

Example using AdamW's step in PyTorch vs fastai:

https://youtu.be/8SF_h3xF3cE?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU&t=1903

Why PyTorch is more popular than TensorFlow nowadays?

- Ease of use (“you can do anything that PyTorch does in TensorFlow. It will just take you twice as much effort to write the code.”)
- PyTorch has more models available:



“With HuggingFace, engineers can use large, trained and tuned models and incorporate them in their pipelines with just a few lines of code. However, a staggering **85% of these models can only be used with PyTorch.**”

<https://www.assemblyai.com/blog/pytorch-vs-tensorflow-in-2023/>

Why PyTorch is more popular than TensorFlow nowadays?

- TensorFlow is not dead!
 - It has a **better deployment infrastructure for putting applications into production** (servers, mobile services, etc.): TensorFlow Serving, TensorFlow Lite.
 - **It can be used with JavaScript, Java, and C++**. Support is also being developed for Julia, Rust, Scala, and Haskell, among others.
 - PyTorch, on the other hand, is very focused on Python.

<https://thenextweb.com/news/why-tensorflow-for-python-is-dying-a-slow-death>

<https://opencv.org/blog/pytorch-vs-tensorflow/>

<https://www.freecodecamp.org/news/pytorch-vs-tensorflow-for-deep-learning-projects/>

First steps

- Bird classification
 - <https://www.kaggle.com/code/jhoward/is-it-a-bird-creating-a-model-from-your-own-data> (you will need internet access to run this Notebook → activate *SMS account verification*)
 - Presentation by Jeremy Howard:
https://youtu.be/8SF_h3xF3cE?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU&t=2314
- Bear classification
 - https://github.com/fastai/fastbook/blob/master/02_production.ipynb
- Imagenette images classification
 - <https://docs.fast.ai/tutorial.imagenette.html>

First steps

- Be sure that you are using GPU acceleration!

Editar Ver Insertar Entorno de ejecución Her

Deshacer Ctrl+M Z

Rehacer Ctrl+Shift+Y

Seleccionar todas las celdas Ctrl+Shift+A

Cortar celda o selección

Copiar celda o selección

Pegar

Eliminar celdas seleccionadas Ctrl+M D

Buscar y reemplazar Ctrl+H

Buscar siguiente Ctrl+G

Buscar anterior Ctrl+Shift+G

Configuración del cuaderno

Borrar todos los resultados

Configuración del cuaderno

Tipo de entorno de ejecución

Python 3

Acelerador por hardware ?

☐ CPU ☒ T4 GPU ☐ A100 GPU ☐ L4 GPU ☐ TPU v2-8

¿Quieres acceder a GPUs premium? [Compra unidades de computación adicionales](#)

☐ Ejecutar automáticamente la primera celda o sección en cualquier ejecución

☐ Omitir resultado de las celdas de código al guardar este cuaderno

Cancelar Guardar

Data loading and manipulation

- DataBlock and DataLoaders

Key questions we want to answer to convert our data into a *DataLoaders* object:

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')])  
dls.dataloaders(path, bs=32)
```

What kind of data are we working with?

Where can we get the examples from?

How can we have a validation set?

Where do we get the labels from?

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')]  
)  
dls.dataloaders(path, bs=32)
```

The inputs for our model will be images (*ImageBlock*) and the output are categories (*CategoryBlock*), such as “bird” or “forest”

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')]  
)  
dls.dataloaders(path, bs=32)
```

We retrieve the images using the *get_image_files* function, which returns a list with all the images in *path* (https://docs.fast.ai/data.transforms.html#get_image_files)

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')]  
)  
dls.dataloaders(path, bs=32)
```

We split the data randomly between training and validation (20%). We set the random seed to partition the data always in the same way.

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label, ←  
    item_tfms=[Resize(192, method='squish')]  
) .dataloaders(path, bs=32)
```

The labels (desired outputs) are obtained from the name of the parent directory of each file (i.e. the name of the folder they are in) (https://docs.fast.ai/data.transforms.html#parent_label)

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')]  
)  
dls.dataloaders(path, bs=32)
```

On each example (*item*) a series of transformations is applied: *resize* to 192x192 pixels, and “*squishing*” (as opposed to “*cropping*”)

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')]  
) .dataloaders(path, bs=32)
```

An interesting transformation is *RandomResizedCrop(size,min_scale)*, which picks random crops that include at least *min_scale*% of the original image, and resizes to *size*.

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(  
    blocks=(ImageBlock, CategoryBlock),  
    get_items=get_image_files,  
    splitter=RandomSplitter(valid_pct=0.2, seed=42),  
    get_y=parent_label,  
    item_tfms=[Resize(192, method='squish')])  
dls.dataloaders(path, bs=32)
```

You could also specify `num_workers=os.cpu_count()`

Number of CPU parallel subprocesses to prepare data before sending it to GPU!

What do you think is faster: to train with larger or smaller batch sizes? 🤔

All this process will be performed on the images/folders in *path*. And the images will be loaded in batches (*bs*) of 32.

DataLoaders documentation:
<https://docs.fast.ai/data.load.html>

Data block tutorial:
<https://docs.fast.ai/tutorial.datablock.html>

Data loading and manipulation

- Data normalization
 - One of the most common and recommended data transformations.
 - Inside the datablock:

```
batch_tfms= Normalize.from_stats(*imagenet_stats)
```

```
batch_tfms= Normalize()
```

```
batch_tfms= Normalize.from_stats(mean, std)
```

Note: “when using a pretrained model through *vision_learner*, the fastai library automatically adds the proper **Normalize transform**; the model has been pretrained with certain statistics in Normalize (usually coming from the ImageNet dataset), so the library can fill those in for you. Note that this only applies with pretrained models” (https://github.com/fastai/fastbook/blob/master/07_sizing_and_tta.ipynb)

Data loading and manipulation

```
pets = DataBlock(blocks=(ImageBlock, CategoryBlock),  
                 get_items=get_image_files,  
                 splitter=RandomSplitter(),  
                 get_y=Pipeline([attrgetter("name"), RegexLabeller(pat = r'^(.*)_d+.jpg$')]),  
                 item_tfms=Resize(128),  
                 batch_tfms=aug_transforms())
```

<https://docs.fast.ai/tutorial.datablock.html>

Standard set of augmentations that generally work pretty well.

https://github.com/fastai/fastbook/blob/master/02_production.ipynb

- 1) **item_tfms** is applied to each individual image before it is copied to the GPU. It resizes all images to same size.
- 2) **batch_tfms** is applied to a batch all at once on the GPU (it's fast).

https://github.com/fastai/fastbook/blob/master/05_pet_breeds.ipynb

Data loading and manipulation

- DataBlock and DataLoaders
 - There are different DataLoaders depending on the type of data you want to deal with and the problem you face:
 - ImageDataLoaders
 - SegmentationDataLoaders
 - LMDataLoader
 - TextDataLoader
 - TabularDataLoaders

Data block tutorial:

<https://docs.fast.ai/tutorial.datablock.html>

DataLoaders documentation:

<https://docs.fast.ai/data.load.html>

Creating a model from scratch

- Simple example:
 - Our input images are 32x32x3
 - We want a network with (in this order):
 - Convolutional layer with 5 7x7 filters, no padding and stride=1. ReLU activation function.
 - 2x2 MaxPooling.
 - Fully-connected layer with 15 neurons and Softmax activation function (with 15 output classes).

Creating a model from scratch

```
simpleNet = sequential(  
    nn.Conv2d(in_channels=3,out_channels=5,kernel_size=(7,7)),  
    nn.ReLU(),  
    nn.MaxPool2d(kernel_size=(2,2)),  
    nn.Flatten(),  
    nn.Linear(in_features=845, out_features=15),  
    nn.Softmax()  
)
```

Here we have a volume of 26x26x5

Here we have a volume of 13x13x5,
i.e. 845 elements

Is this really necessary?? Carefully check the documentation.

Creating a model from scratch

The *Learner* object includes the model (*simpleNet*), the data (*dls*) and the loss function (*loss_func*). We have everything to train our model.

```
learn = Learner(dls, simpleNet, loss_func=CrossEntropyLossFlat(), metrics=accuracy)
```

data

model

loss function

<https://docs.fast.ai/losses.html>

<https://pytorch.org/docs/stable/nn.html#loss-functions>

Metric for assessing performance

<https://docs.fast.ai/metrics.html>

Creating a model from scratch

```
learn.summary()
```

Sequential (Input shape: 32 x 3 x 32 x 32)

Layer (type)	Output Shape	Param #	Trainable
Conv2d ReLU	32 x 5 x 26 x 26	740	True
MaxPool2d	32 x 5 x 13 x 13		
Flatten	32 x 845		
Linear Softmax	32 x 15	12690	True

Total params: 13,430
Total trainable params: 13,430
Total non-trainable params: 0

Batch size

`learn.summary()` allows you to verify that you have built the architecture correctly

5 filters with $7 \times 7 \times 3 + 1$ (bias) parameters to learn

$845 \times 15 + 15$ (bias) parameters to learn.

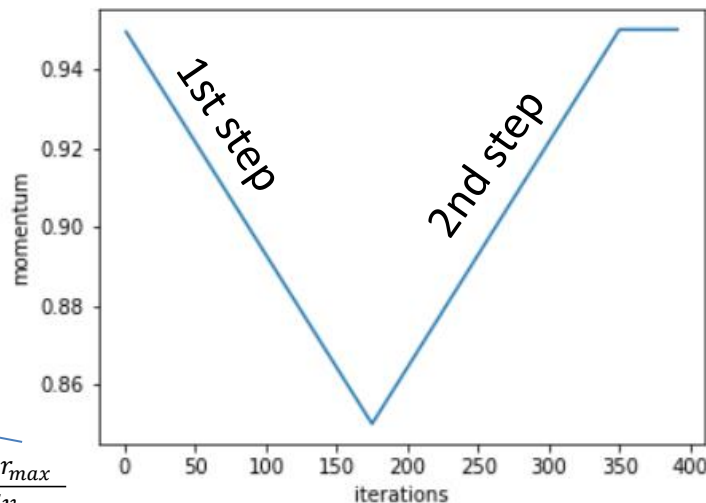
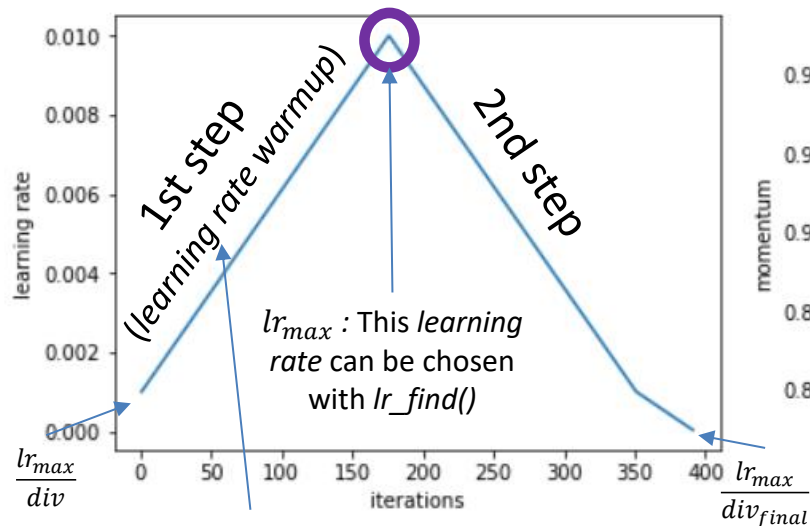
Note that **most of the weights are in the fully-connected!** Be careful not to include too many layers of this type when you create your own architecture!

Training

- `learn.fit(n_epoch)`
 - Trains the model for a certain numbers of epochs
- `learn.fit_one_cycle(n_epoch)`
 - Trains the model for a certain number of epochs using the *1cycle policy* of Leslie N. Smith (<https://arxiv.org/abs/1708.07120>)

Training

- `learn.fit_one_cycle(n_epoch)`
 - Trains the model for a certain number of epochs using the *1cycle policy* of Leslie N. Smith (<https://arxiv.org/abs/1708.07120>)



This combined strategy allows you to train much faster (*super-convergence*)

Stabilizes training during the early optimization phase when using large batch sizes (see [Goyal 2017](#))

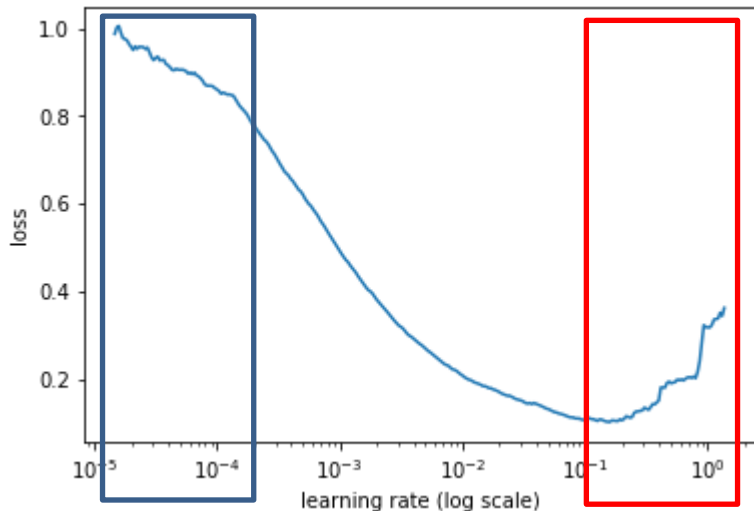
<https://derekchia.com/the-1-cycle-policy/>

<https://www.youtube.com/watch?v=CJKNdu2dxOE&t=7203s>

Note: it's not 1 cycle per epoch, but 1 cycle for the whole training! If you want multiple cycles, check the [cyclical learning rates paper](#).

Training

- `learn.fit_one_cycle(n_epoch)`
 - The maximum *learning rate* used in the *1cycle policy* is chosen with the *Learning Rate Finder*: `learner.lr_find()`



It uses an epoch to build a graph like the one on the left. It helps us to choose a *learning rate* not **too big** or **too small**.

We want to choose a *learning rate* as large as possible (without making the training diverge) to train/optimize as fast as possible.

<https://sgugger.github.io/how-do-you-find-a-good-learning-rate.html>

Training

- Multiple calls to *fit* or *fit_one_cycle*
 - like Keras, if these functions are called several times, we'd be training the model incrementally from the point/weights obtained from the previous call.
- Interesting possibility (with pre-trained models): *discriminative learning rates*  

If you start with an uninitialized network, that you want to train from scratch, would it make sense to use it?

 - In the training function, use *slice()* to indicate the *learning rate*.
 - Example: `learn.fit_one_cycle(3, lr_max=slice(1e-5, 1e-3))`

The head will train with 1e-3 and in previous layers will use smaller learning rates (with 3 layer groups, we'd use, as lr_max, 1e-5 in the first layer group and 1e-4 in the second one).

Training

- You can define, initialize and use different optimizers
 - <https://docs.fast.ai/optimizer.html>
 - Example using Adagrad
(<https://pytorch.org/docs/stable/generated/torch.optim.Adagrad.html>)

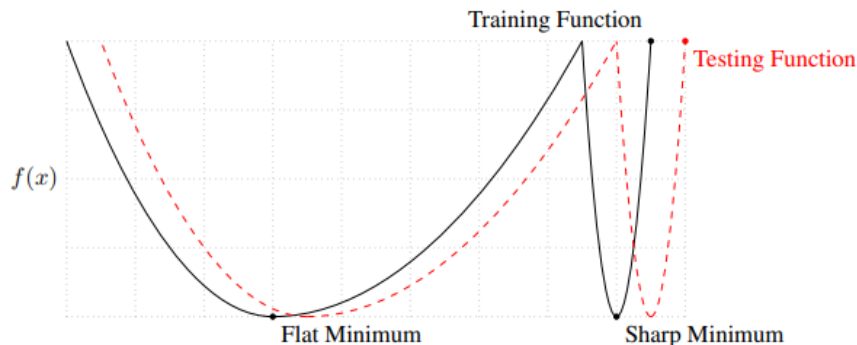
```
learn = Learner(dls, yourNetwork, metrics=accuracy,  
               loss_func=LabelSmoothingCrossEntropyFlat(),  
               opt_func=partial(OptimWrapper,  
                               opt=torch.optim.Adagrad,  
                               lr=0.0001, lr_decay=0.01,  
                               weight_decay=0.1))
```

Training

- Pay attention to the relationship between different parameters: batch size (bs), learning rate (lr), weight decay (wd), momentum
 - small lr favors overfitting → large values should be used (but avoiding too large values that may prevent from converging).
 - small bs regularizes (it appears to locate minima with better generalization properties) → in small models perhaps better use big bs .
 - wd should be set to small values (the larger the value, the more regularization (higher *large weights* penalization)), otherwise it will be difficult to fit the data.
 - if we have a large lr and also a high momentum we run the risk of not converging.

Training

- Relevant concepts:
 - Sharp vs Flat Minima / Optimization of our loss function vs Generalization ability
 - <https://www.inference.vc/sharp-vs-flat-minima-are-still-a-mystery-to-me/>
 - <https://towardsdatascience.com/what-can-flatness-teach-us-understanding-generalisation-in-deep-neural-networks-a7d66f69cb5c>
 - Zhou et al. (2020). Towards theoretically understanding why SGD generalizes better than Adam in deep learning. *Advances in Neural Information Processing Systems*, 33, 21285-21296.
 - Hochreiter, S., & Schmidhuber, J. (1997). Flat minima. *Neural computation*, 9(1), 1-42.
 - Keskar et al. (2016). On large-batch training for deep learning: Generalization gap and sharp minima. arXiv preprint arXiv:1609.04836.



Prediction

- Once we have trained our model we can:

- Perform prediction on a single example: `learn.predict(example)`
- Perform prediction on a set of examples (test):

```
test_dl = learn.dls.test_dl(files_test, with_labels=True, device='cuda')  
preds, targs = learn.get_preds(dl=test_dl)
```

Otherwise, it can be much slower!

- It is key to scale/normalize the test data following exactly the same protocol used in training (using the same mean and std)
 - This is done automatically for you by fastai, using `learn.get_preds` or `learn.dls.test_dl`.
 - <https://forums.fast.ai/t/do-we-need-to-normalize-single-image-before-running-predict-function-on-it/44301/3>
 - <https://forums.fast.ai/t/99-accuracy-on-valid-data-1-accuracy-on-test-data-what-am-i-missing/80408/2>

Interpretation of results

- `interp = ClassificationInterpretation.from_learner(learn)`
- `interp.plot_confusion_matrix(figsize=(12, 12), title='Title')`
- `interp.most_confused(min_val=10)`
- `interp.plot_top_losses(10, nrows=2, figsize=(32, 4))`

And many other possibilities in <https://docs.fast.ai/interpret.html>

Saving and loading models

To know where it was saved: `learn.path`

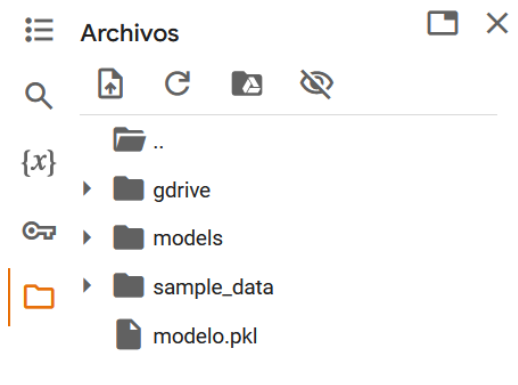
Or check the folders browser:

```
learn.export('modelo.pkl')
```

```
from fastai.vision.all import load_learner  
loaded_learn = load_learner('modelo.pkl')
```

Once you have loaded the model, you can perform inference with it:

```
loaded_learn.predict(files_test[0])
```



Everything will be in the .pkl file within the `learn.path` folder. If you deploy your model on a different machine, this is the file you'll need to copy.

Transfer Learning: Loading an existing model

- fastai integrates numerous trained state-of-the-art models:
 - <https://timm.fast.ai/>
 - <https://github.com/huggingface/pytorch-image-models>
- **Idea:** take something that already works well for a similar problem and reuse it in another problem (*transfer learning*).

Transfer Learning: Loading an existing model

```
model = fastai.vision.models.resnet18
learn = vision_learner(dls, model)
learn.summary()
```

We load the pre-trained model

We use *vision_learner* (https://docs.fast.ai/vision_learner.html): “All the functions necessary to build *Learner* suitable for transfer learning in computer vision”

We explore the architecture, as well as its trainable parameters

cnn_learner: Deprecated name for *vision_learner* – **do not use it**

Transfer Learning: Loading an existing model

- Does the above contain the weights?
 - By default, yes (*pretrained=True*). See <https://docs.fast.ai/vision.learner.html>
- How to modify the last layers of a pre-trained model?

```
custom_head = nn.Sequential(  
    nn.AvgPool2d((7,7)),  
    nn.Flatten(),  
    nn.Linear(512, 200))
```

```
learn = vision_learner(dls, resnet18, custom_head=custom_head)
```

Fine-tuning

- `learn.fine_tune(epochs, freeze_epochs)`
 - 1) Trains the added layers (*head*) for one epoch (by default, `freeze_epochs=1`), with all other layers “frozen”.
 - 2) “Unfreezes” all layers, and trains them for the indicated number of epochs.

`fine_tune()` internally uses `fit_one_cycle()` with discriminative learning rates in the second phase.

See <https://github.com/fastai/fastai/blob/master/fastai/callback/schedule.py#L161>

Fine-tuning

- If you want to “freeze” part of a model, so no changes happen to these weights (i.e. they’re not trained):

```
# We freeze all weights in the model
for param in model.parameters():
    param.requires_grad = False
```

<https://stackoverflow.com/questions/51748138/pytorch-how-to-set-requires-grad-false>

- You can also use **`learner.freeze()`**, **`learner.unfreeze()`**, **`learner.freeze_to()`**. See <https://docs.fast.ai/learner.html> and <https://www.kaggle.com/code/danielliao/understanding-learner-freeze-to>

Fine-tuning

- If you use `pretrained=False` in `vision_learner` all layers will be trainable.
- If you use `pretrained=True`, all layers will be non trainable (except your new header and the batchnorm layers).
 - If you want batchnorm layers to be non trainable: `train_bn=False`

Problems with Colab?

- General advice: **even if this year we have Google Colab Pro licenses, please, use Colab resources judiciously.**
 - Otherwise, RAM problems may arise or you may be temporarily blocked/restricted from using the GPUs.
- If you employ good coding practices, and reasonable experiments are carried out in an orderly manner, there should be no problem.

General advice

- “It's only by practicing (and failing) a lot that you will get an intuition of how to train a model.”
- Do not hesitate in consulting the online help directly in the Notebook:

??function

doc(function)

??learn.fine_tune

doc(learn.fine_tune)

Computer Vision: Assignment 2

Deep Learning for Computer Vision

Pablo Mesejo and Siham Tabik

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA

